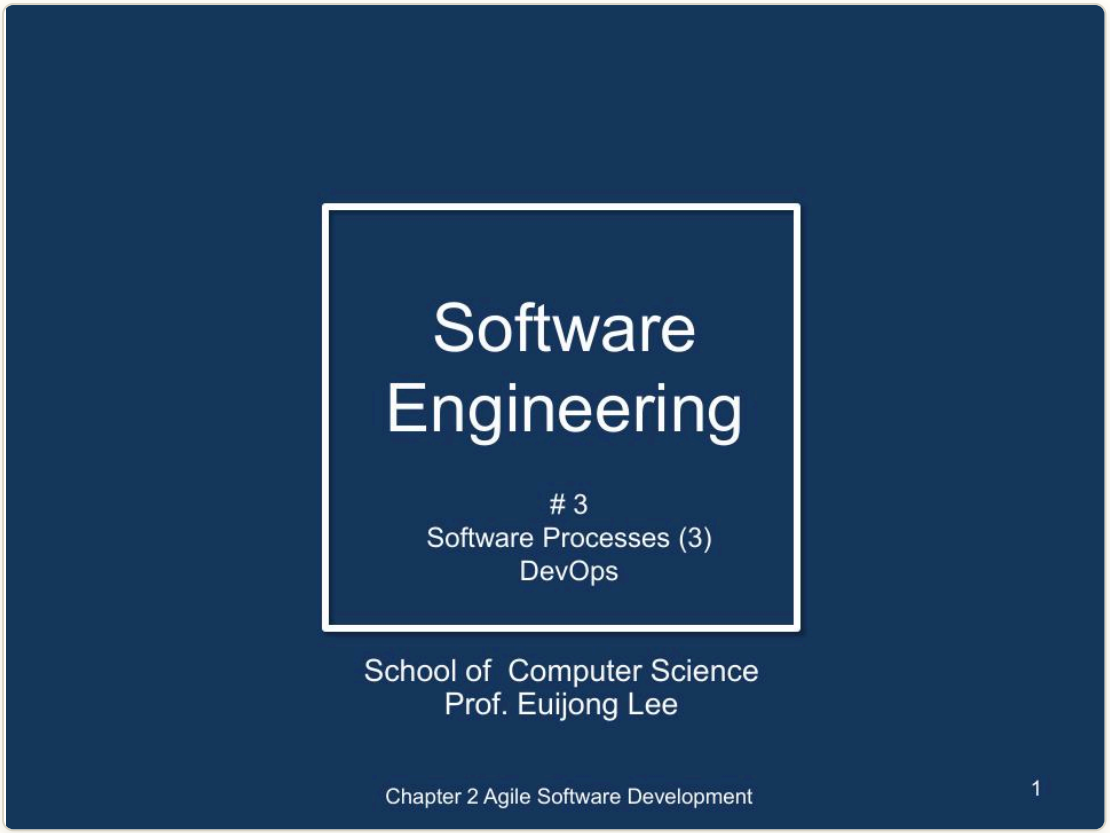
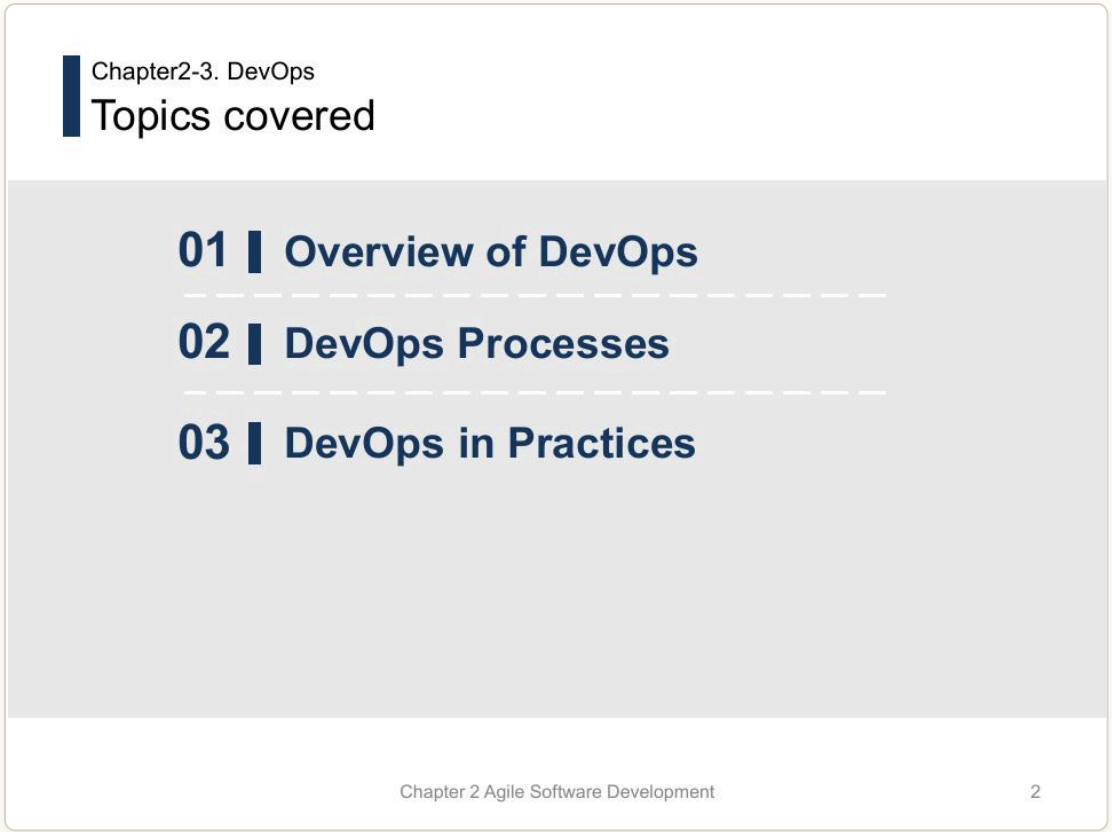


소프트웨어공학 #3 — 소프트웨어 프로세스 (3): 데브옵스

- 이의종 교수님

✍️ 메모

💡 이해 29쪽 짧은 장 — 정의 · 원칙 4 · 장점 4 · 프로세스 8단계 · 애자일과 비교가 핵심

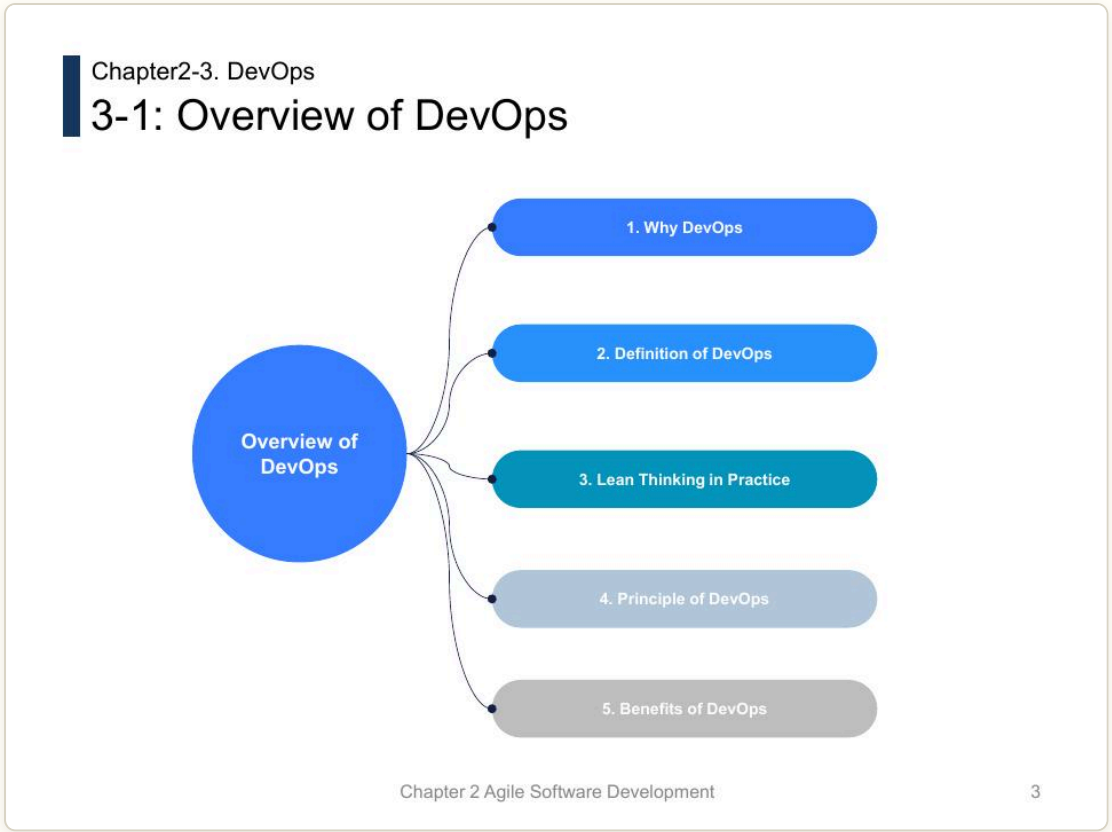


다루는 내용

- 데브옵스 개요 / 데브옵스 프로세스 / 실무의 데브옵스

✍ 메모

— 넘어가도 되는 쪽



3-1: 데브옵스 개요

- 1. 왜 데브옵스인가
- 2. 데브옵스의 정의
- 3. 린(Lean) 사고
- 4. 데브옵스의 원칙
- 5. 데브옵스의 장점

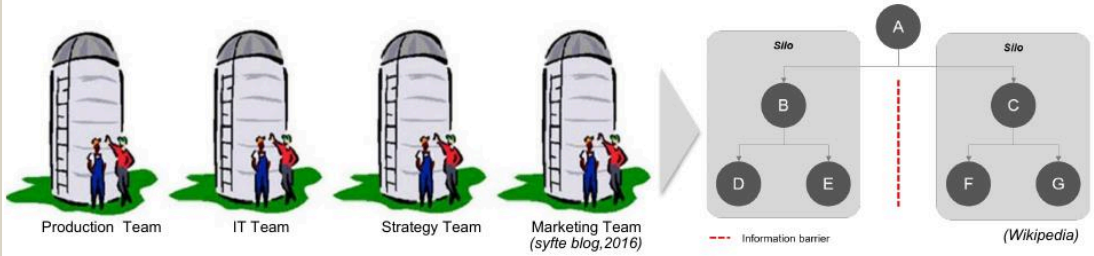
✍️ 메모

💡 이해 절 표지 — 아래 쪽들의 지도

Overview of DevOps

Why DevOps?

- **Silos based development process**
  - After each stage of software development is completed, proceed to the next
  - Absence of interaction between each step



Chapter 3 DevOps 4

왜 데브옵스? — 사일로 기반 개발

- **사일로(silo) 기반 개발 프로세스**
  - SW 개발의 각 단계가 끝나야 다음으로
  - 각 단계 사이에 **상호작용이 없다**
- 그림: 생산팀·IT팀·전략팀·마케팅팀이 각자 곡물 저장고(사일로)에 갇혀 있고, 사일로 사이엔 **정보 장벽**

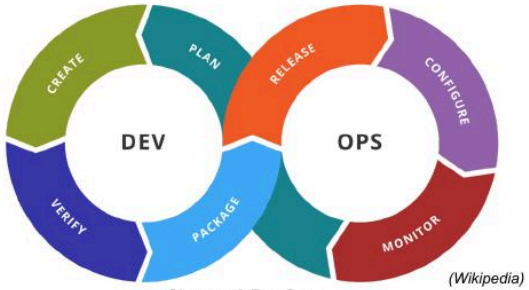
✍ 메모

★ 시험 데브옵스가 풀려는 문제 = 팀 간 사일로(단절)

Overview of DevOps

Why DevOps?

- **DevOps?**
  - **DevOps** is a software development methodology using iterative mechanism
  - Interconnect Development(Dev), Operations(Ops), and user feedbacks
  - Based on Agile and Lean methods
  - To solve insufficient communication between development team, development delay, and other problems in projects during operation



(Wikipedia)

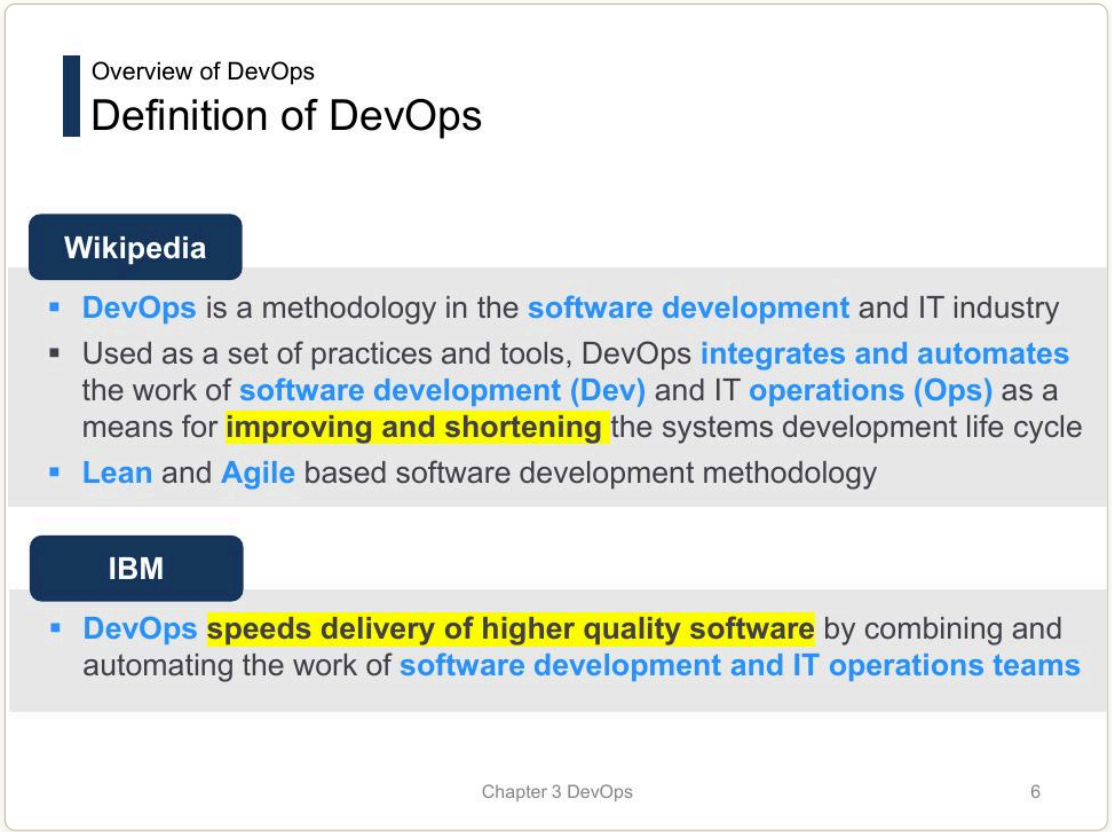
Chapter 3 DevOps 5

왜 데브옵스? — 데브옵스란

- **반복적 메커니즘**을 쓰는 SW 개발 방법론
- **개발(Dev)**, **운영(Ops)**, **사용자 피드백**을 서로 연결
- **애자일과 린(Lean)** 방법에 기반
- 운영 중 프로젝트에서 생기는 개발팀 간 **소통 부족, 개발 지연** 등의 문제를 해결하려고

✍ 메모

★ 시험 **Dev + Ops + 사용자 피드백 연결, 애자일 + 린 기반**

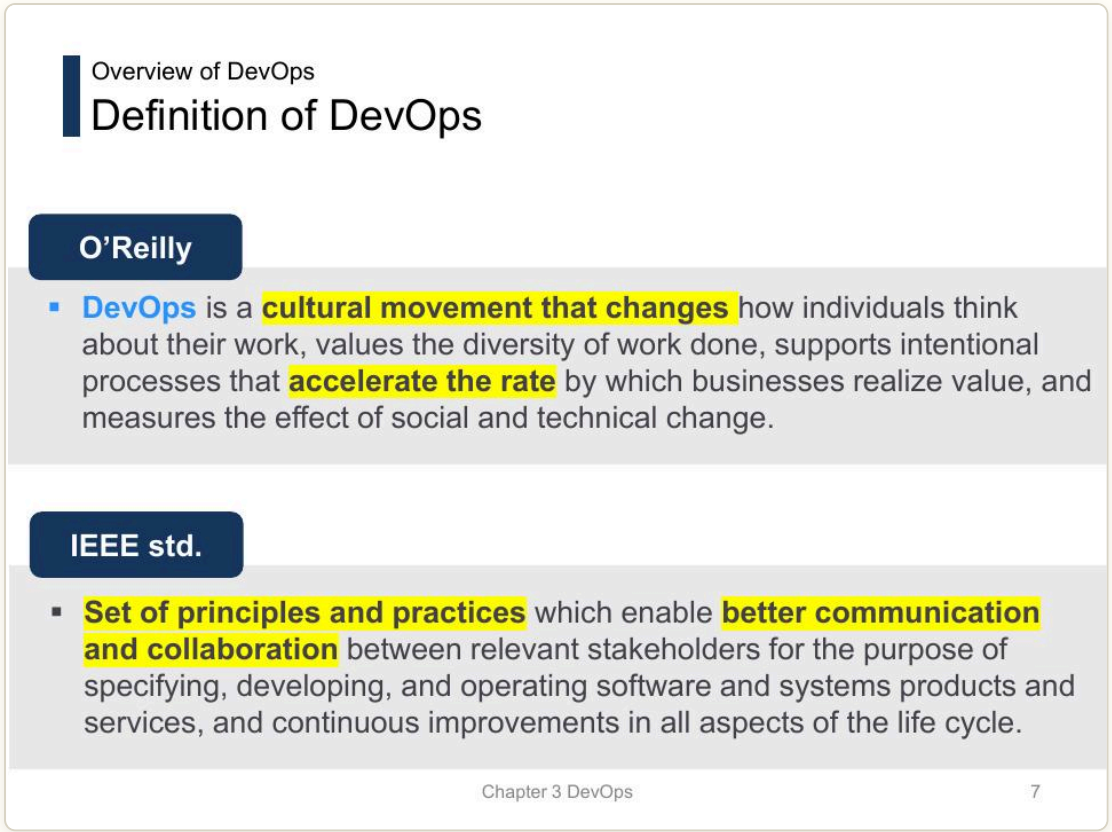


데브옵스의 정의 ① 위키백과 · IBM

- **위키백과**
  - SW 개발·IT 업계의 방법론
  - 실천과 도구의 집합으로서, **개발(Dev)과 IT 운영(Ops)의 일을 통합·자동화**해 시스템 개발 생명주기를 개선·단축
  - 린과 애자일 기반 SW 개발 방법론
- **IBM**
  - SW 개발팀과 IT 운영팀의 일을 **결합·자동화**해 더 높은 품질의 SW를 **더 빨리** 전달

✍ 메모

— 넘어가도 되는 쪽



데브옵스의 정의 ② O'Reilly · IEEE

- **O'Reilly** — 데브옵스는 **문화 운동(cultural movement)**
  - 개인이 자기 일을 생각하는 방식을 바꾸고, 일의 다양성을 존중하고, 기업이 가치를 실현하는 속도를 높이는 의도적 프로세스를 지원하고, 사회적·기술적 변화의 효과를 측정
- **IEEE 표준** — 관련 이해관계자 사이의 **더 나은 소통과 협업**을 가능하게 하는 **원칙과 실천의 집합**
  - 목적: SW·시스템 제품과 서비스를 명세·개발·운영하고, 생명주기 모든 측면을 **지속적으로 개선**

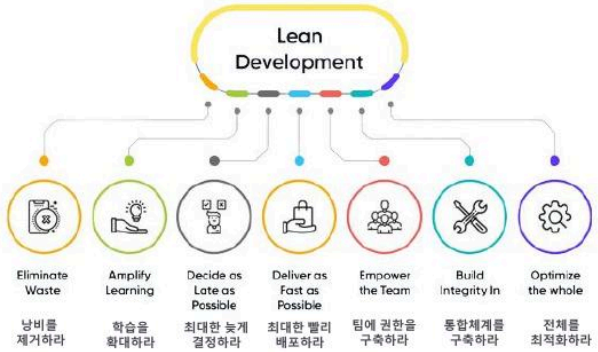
✍️ 메모

💡 이해 정의마다 강조점: 위키·IBM = 통합·자동화 / O'Reilly = 문화 / IEEE = 소통·협업

Overview of DevOps

Lean Thinking

- **Lean Software Development (LSD)**
  - **"Think Big, Act Small, Fail Fast, Learn Rapidly"**
  - Lean development is a methodology that was first applied to manufacturing, but is now being applied to management, UX, startups, and to software development
  - Lean software development is the process of optimizing how value is created by reducing waste



(https://www.netsolutions.com/insights/7-principles-of-lean-software-development/)

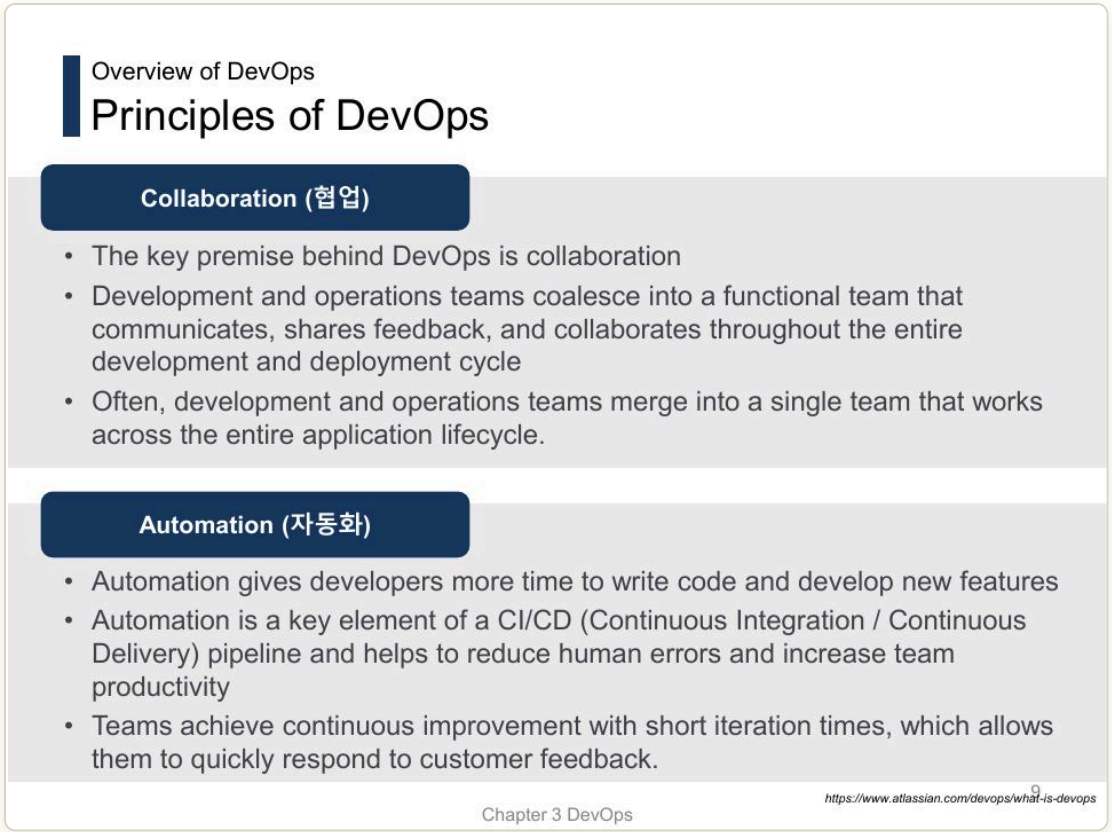
린 사고 (Lean Thinking)

- **린 SW 개발 (LSD)**
  - 구호: **"Think Big, Act Small, Fail Fast, Learn Rapidly"**
  - 처음엔 **제조업**에 쓰였고 이제 관리·UX·스타트업·SW 개발에도 쓰임
  - **낭비를 줄여** 가치 창출 방식을 최적화하는 과정
- **린 7원칙**
  - 낭비를 제거하라 / 학습을 확대하라 / 최대한 늦게 결정하라 / 최대한 빨리 배포하라
  - 팀에 권한을 부여하라 / 통합체계를 구축하라 / 전체를 최적화하라

✍️ 메모

★ 시험 린 핵심 = 낭비 제거 (reducing waste)

⚠ 주의 '늦게 결정' + '빨리 배포' 짝이 헷갈리기 쉬움 — 결정은 늦게, 배포는 빨리



데브옵스 원칙 ① 협업 · 자동화

- **Collaboration (협업)** — 데브옵스의 **핵심 전제**
  - 개발팀과 운영팀이 하나의 기능 팀으로 뭉쳐 개발·배포 주기 내내 소통·피드백·협업
  - 흔히 애플리케이션 생명주기 전체를 맡는 **단일 팀으로 합쳐짐**
- **Automation (자동화)**
  - 개발자가 코드와 새 기능에 쓸 시간이 늘어난다
  - **CI/CD (지속적 통합/지속적 전달) 파이프라인**의 핵심 요소 — 사람의 실수를 줄이고 생산성↑
  - 짧은 반복으로 지속적 개선 → 고객 피드백에 빠르게 대응

✍️ 메모

★ 시험 **CI/CD = Continuous Integration / Continuous Delivery**

Overview of DevOps

Principles of DevOps

Continuous Improvement (지속적 개선)
Continuous integration and delivery (지속적 통합/배포)

- Continuous improvement was established as a staple of agile practices, as well as lean manufacturing and Improvement Kata.
  - Kata? *Improvement Kata is a method where team leaders and members continually practice a kata routine that develops and channels their abilities to solve problems.*
- It's the practice of focusing on experimentation, minimizing waste, and optimizing for speed, cost, and ease of delivery.
- Continuous improvement is also tied to continuous delivery, allowing DevOps teams to continuously push updates that improve the efficiency of software systems.
- The constant pipeline of new releases means teams consistently push code changes that eliminate waste, improve development efficiency, and bring more customer value.



https://www.atlassian.com/devops/what-is-devops

10

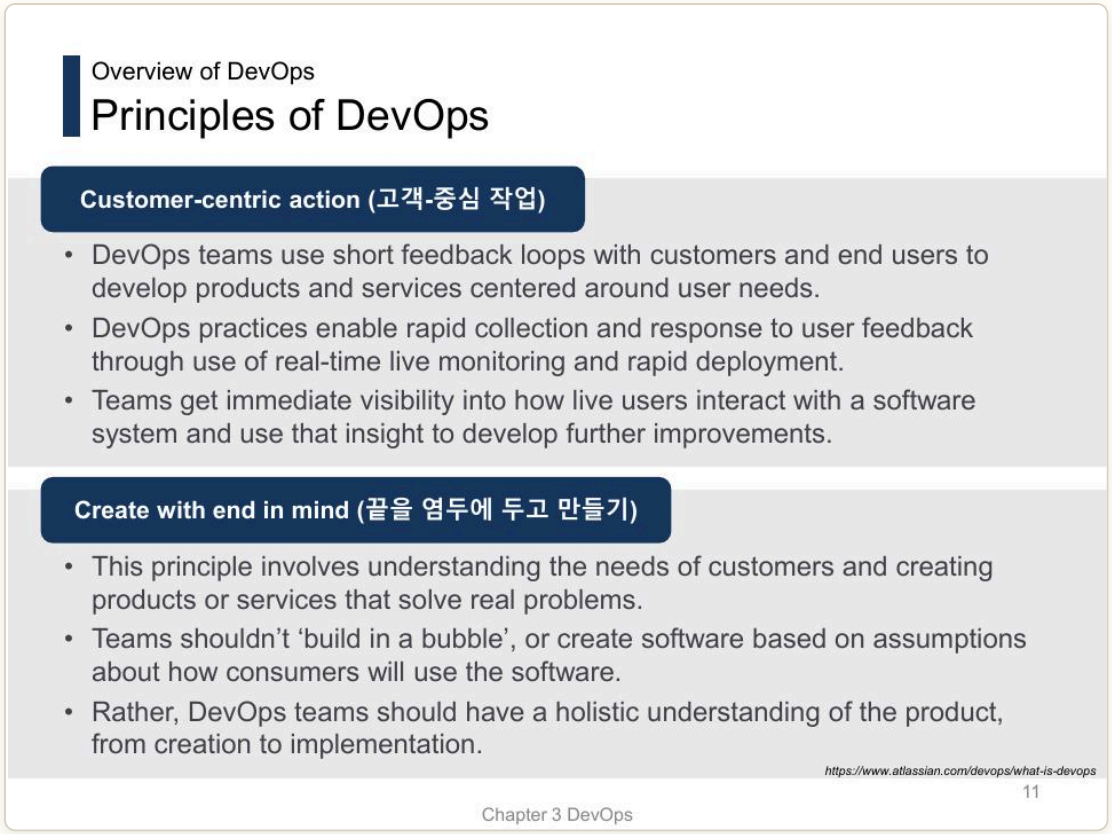
Chapter 3 DevOps

데브옵스 원칙 ② 지속적 개선

- **Continuous Improvement (지속적 개선)**
  - 애자일 실천, 린 제조, **Improvement Kata**의 기본으로 자리 잡음
    - Kata = 팀 리더와 팀원이 문제 해결 능력을 키우는 정해진 루틴(카타)을 계속 연습하는 방법
  - 실험에 집중하고, 낭비를 줄이고, 속도·비용·전달 용이성을 최적화하는 실천
  - **지속적 전달**과 연결 — 시스템 효율을 높이는 업데이트를 계속 내보냄
  - 끊임없는 릴리스 파이프라인 → 낭비 제거, 개발 효율↑, 고객 가치↑

✍ 메모

— 넘어가도 되는 쪽

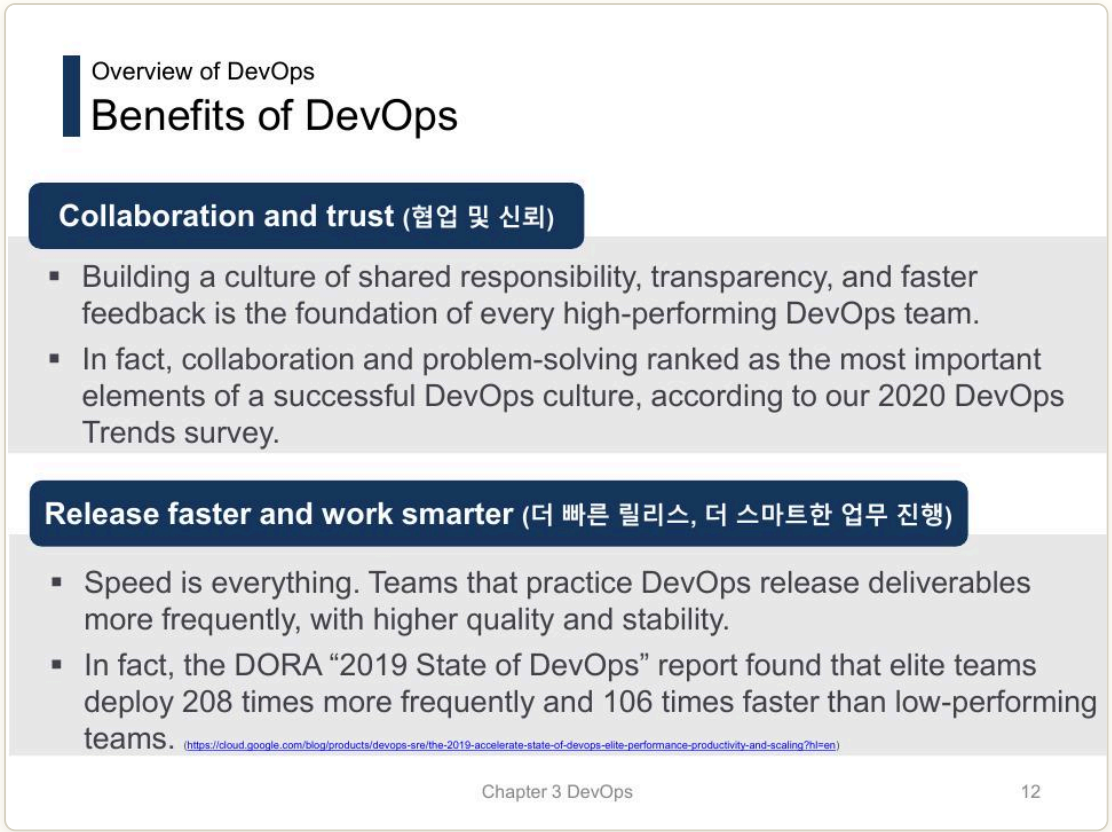


데브옵스 원칙 ③ 고객 중심 · 끝을 염두에

- **Customer-centric action (고객 중심 작업)**
  - 고객·최종 사용자와 **짧은 피드백 루프**로 사용자 요구 중심의 제품 개발
  - **실시간 모니터링과 빠른 배포**로 사용자 피드백을 빨리 모으고 대응
  - 실제 사용자가 시스템을 어떻게 쓰는지 즉시 보고, 그 통찰로 개선
- **Create with end in mind (끝을 염두에 두고 만들기)**
  - 고객의 요구를 이해하고 **실제 문제를 푸는** 제품을 만든다
  - **'거품 속에서 만들기'** 금지 — 사용 방식을 가정만 하고 만들지 말 것
  - 생성부터 구현까지 제품을 **총체적으로** 이해

✍️ 메모

★ 시험 원칙 5개: 협업 · 자동화 · 지속적 개선(+CI/CD) · 고객 중심 · 끝을 염두에

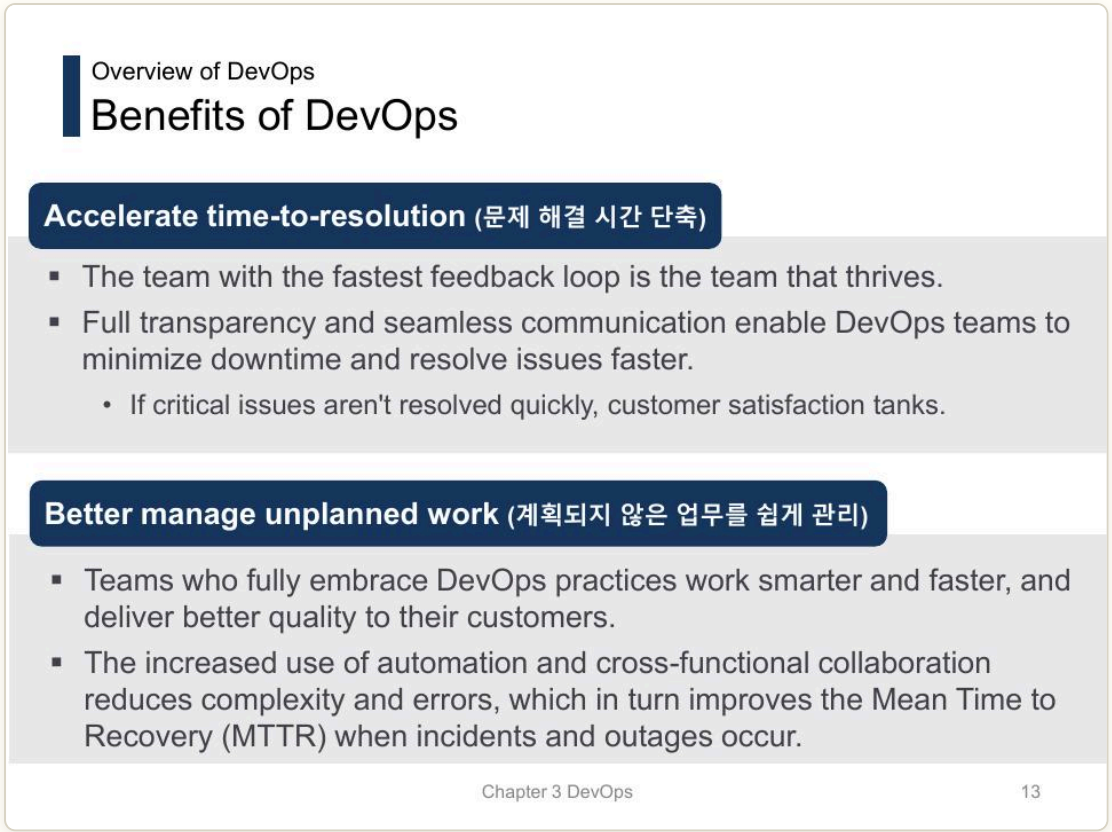


데브옵스의 장점 ① 협업·신뢰 · 빠른 릴리스

- **Collaboration and trust (협업과 신뢰)**
  - 공동 책임·투명성·빠른 피드백 문화가 고성과 데브옵스 팀의 기반
  - 2020 DevOps Trends 조사: **협업과 문제 해결**이 성공적인 데브옵스 문화의 가장 중요한 요소
- **Release faster and work smarter (더 빠른 릴리스, 더 스마트한 업무)**
  - 속도가 전부 — 더 자주, 더 높은 품질과 안정성으로 릴리스
  - DORA 2019 보고서: 엘리트 팀은 저성과 팀보다 **208배 더 자주**, **106배 더 빨리** 배포

✍ 메모

✏ 예·풀이 숫자 208배(빈도) · 106배(속도) — 순서 바꿔 내기 쉬움



데브옵스의 장점 ② 해결 시간 단축 · 계획 외 업무 관리

- **Accelerate time-to-resolution (문제 해결 시간 단축)**
  - 피드백 루프가 가장 빠른 팀이 번성한다
  - 완전한 투명성과 매끄러운 소통으로 다운타임을 줄이고 문제를 빨리 해결
  - 중요한 문제를 빨리 못 풀면 고객 만족이 곤두박질
- **Better manage unplanned work (계획되지 않은 업무를 잘 관리)**
  - 자동화와 교차 기능 협업이 복잡성·오류를 줄여, 사고·장애 시 **MTTR(평균 복구 시간)**을 개선

✍️ 메모

★ 시험 장점 4개: 협업·신뢰 · 빠른 릴리스 · 해결 시간 단축 · 계획 외 업무 관리

💡 이해 MTTR = Mean Time To Recovery

Overview of DevOps

DevOps Application Trend

- **DevOps Application Trend**
  - IT organizations that must provide customer service 24/7 have applied DevOps for software development and operation strategies (e.g., Amazon, Google, and Netflix)

회사	배포 주기	배포 소요 시간	신뢰성	사용자 응답
아마존	23,000/day	minutes	high	high
구글	5,500/day	minutes	high	high
넷플릭스	500/day	minutes	high	high
페이스북	1/day	hours	high	high
트위터	3/week	hours	high	high
일반 회사	1/9 months	months/quarters	low/medium	low/medium

("소프트웨어 공학 이론과 실제", 홍장의, 한빛아카데미)

14

Chapter 3 DevOps

데브옵스 적용 동향

- **연중무휴(24/7) 고객 서비스**가 필요한 IT 조직이 개발·운영 전략으로 데브옵스를 적용 (예: 아마존, 구글, 넷플릭스)
- 표: 회사 / 배포 주기 / 배포 소요 시간 / 신뢰성 / 사용자 응답
  - 아마존 23,000/일 · 분 단위 · high · high
  - 구글 5,500/일 · 분 · high · high
  - 넷플릭스 500/일 · 분 · high · high
  - 페이스북 1/일 · 시간 · high · high
  - 트위터 3/주 · 시간 · high · high
  - 일반 회사 **9개월에 1번** · 몇 달~분기 · low/medium · low/medium

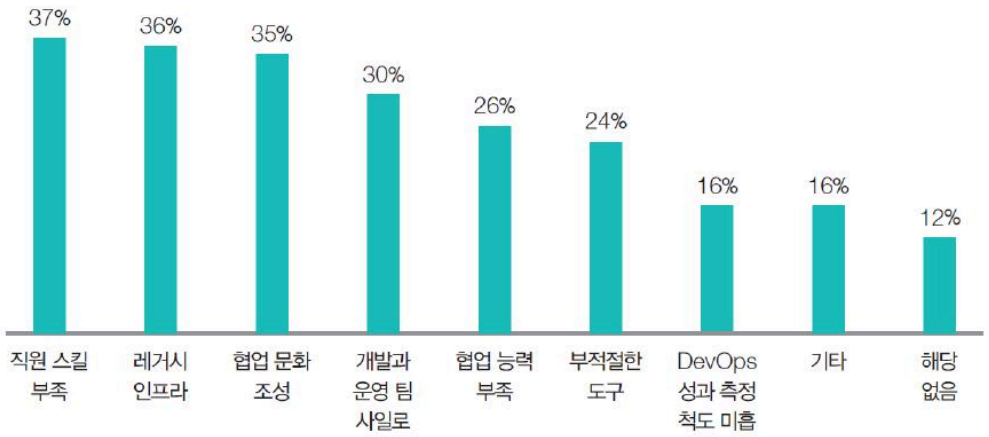
✍️ 메모

💡 이해 요지: 데브옵스 기업은 하루 수천 번 배포, 일반 회사는 9개월에 1번

Overview of DevOps

DevOps Application Trend

▪ **Obstructive factors to apply DevOps**



("소프트웨어 공학 이론과 실제", 홍장의, 한빛아카데미)

15

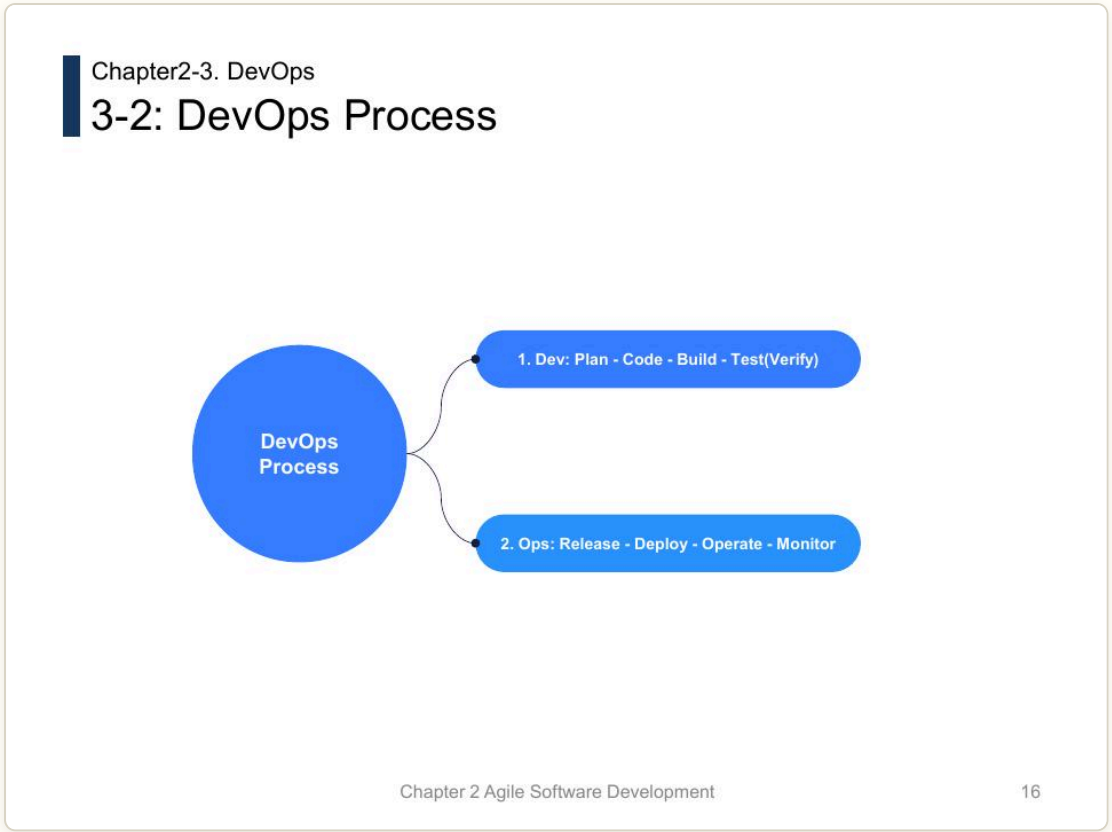
Chapter 3 DevOps

데브옵스 도입 장애 요인 (그래프)

- 직원 스킬 부족 37% > 레거시 인프라 36% > 협업 문화 조성 35% > 개발·운영 팀 사일로 30% > 협업 능력 부족 26% > 부적절한 도구 24% > DevOps 성과 측정 척도 미흡 16% · 기타 16% > 해당 없음 12%

✍ 메모

★ 시험 1위 = 직원 스킬 부족 (도구 문제는 하위권)



3-2: 데브옵스 프로세스

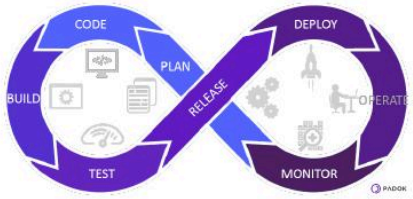
- 1. Dev: Plan → Code → Build → Test(Verify)
- 2. Ops: Release → Deploy → Operate → Monitor

✍️ 메모

💡 이해 절 표지 — 아래 쪽들의 지도

DevOps Process

DevOps Process



- **Plan**
  - This is the part of the project where you are **organizing the tasks, and schedules** and setting up your project management tools.
  - The idea is to plan tasks using the **user story** process from the agile methodology.
  - A perfect user story as a what (who, where, trigger), a why, and acceptance criteria.
- **Code**
  - Doing **code development and code review.**
  - In DevOps practice, it is important to share a code tool between Ops and developers teams like Github or Gitlab.

(https://www.padok.fr/en/blog/devops-process)

17

Chapter 3 DevOps

데브옵스 프로세스 ① Plan · Code

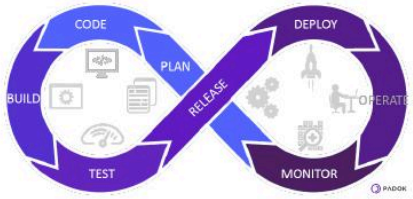
- **Plan (계획)**
  - 태스크와 일정을 정리하고 프로젝트 관리 도구를 설정
  - 애자일의 **사용자 스토리** 과정으로 태스크를 계획
  - 좋은 사용자 스토리 = **무엇**(누가, 어디서, 계기) + **왜** + **인수 기준**
- **Code (코딩)**
  - 코드 개발과 **코드 리뷰**
  - 운영팀과 개발팀이 **GitHub·GitLab 같은 코드 도구를 공유**하는 것이 중요

✍ 메모

— 넘어가도 되는 쪽

DevOps Process

DevOps Process



- **Build**
  - The goal here is to **build the source code** into one desired format, compiling, testing, and deploying in a particular place of the infrastructure.
  - Once this step sets up the **continuous integration (CI) and delivery (CD) tools** can check and verify the source code from Source Code Management and build it.
- **Test (Verify)**
  - The continuous testing process **reduces risks.**
  - **Automatic tests** ensure that no bugs will be implemented in production.
  - You have to implement testing tools in your workflow to ensure the best development quality for your software.

(https://www.padok.fr/en/blog/devops-process)

데브옵스 프로세스 ② Build · Test

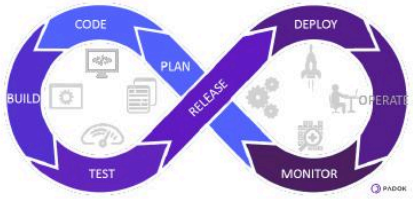
- **Build (빌드)**
  - 소스 코드를 원하는 형식으로 만들기 — 컴파일, 테스트, 인프라의 특정 위치에 배포
  - 이 단계에서 **CI·CD 도구**가 소스 코드 관리(SCM)에서 코드를 확인·검증하고 빌드
- **Test / Verify (테스트·검증)**
  - **지속적 테스트**가 위험을 줄인다
  - **자동 테스트**로 버그가 운영 환경에 들어가지 않게
  - 워크플로에 테스트 도구를 넣어 최고의 개발 품질 확보

✍ 메모

— 넘어가도 되는 쪽

DevOps Process

DevOps Process



- **Release**
  - The code has passed the testing (continuous integration) process and is **ready to be deployed.**
- **Deploy**
  - The operational team is **deploying the new feature in production**.
  - But as automation is one of the DevOps principles, it is possible to set up continuous deployment.
- **Operate / configure infrastructure**
  - The Ops builder **maintain a scalable infrastructure**, infrastructure as code, and check security issues and log management.
- **Monitor**
  - Monitoring is an important step as it allows you to fix incidents faster and create a better experience for your end-user.

(https://www.padok.fr/en/blog/devops-process)

19

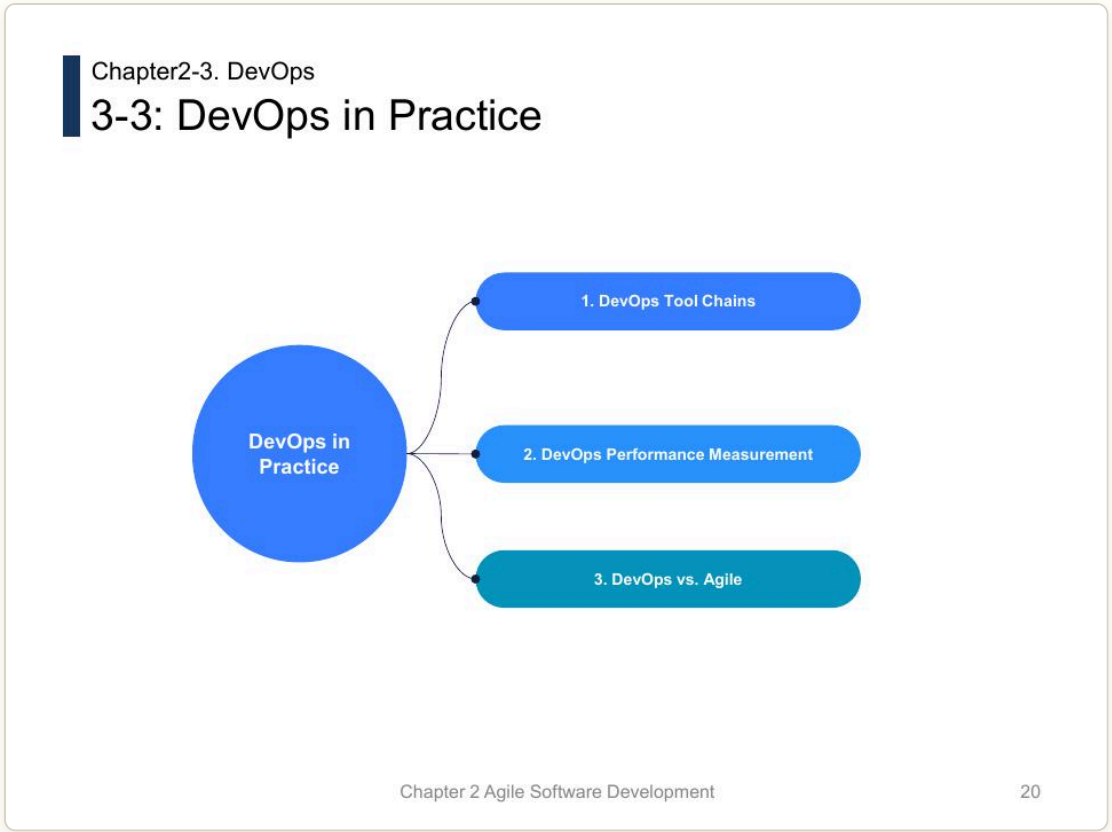
Chapter 3 DevOps

데브옵스 프로세스 ③ Release · Deploy · Operate · Monitor

- **Release (릴리스)** — 코드가 테스트(CI)를 통과해 배포 준비 완료
- **Deploy (배포)** — 운영팀이 새 기능을 운영 환경에 배포
  - 자동화가 원칙이니 **지속적 배포(continuous deployment)** 설정 가능
- **Operate / configure infrastructure (운영·인프라 구성)**
  - 확장 가능한 인프라, **코드형 인프라(IaC)** 유지, 보안 이슈·로그 관리
- **Monitor (모니터링)** — 사고를 빨리 고치고 최종 사용자 경험을 개선

✍ 메모

★ 시험 8단계 순서: Plan → Code → Build → Test | Release → Deploy → Operate → Monitor (앞 4 = Dev, 뒤 4 = Ops) — ∞ 모양 루프



3-3: 실무의 데브옵스

- 1. 데브옵스 툴체인
- 2. 데브옵스 성과 측정
- 3. 데브옵스 vs 애자일

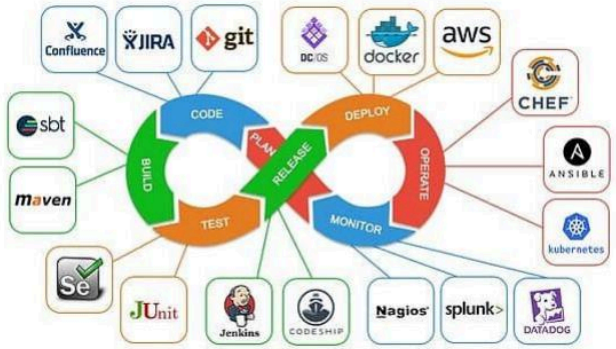
✍️ 메모

💡 이해 절 표지 — 아래 쪽들의 지도

DevOps in Practice

DevOps Tool Chains

- **DevOps Tool chains?**
  - A DevOps toolchain is **a collection of tools**, often from a variety of vendors, that operate as an integrated unit to design, build, test, manage, measure, and operate software and systems.
  - It enables development and operations teams to collaborate across the entire product lifecycle and tackles key DevOps fundamentals including continuous integration, continuous delivery, automation, and collaboration.



21

Chapter 3 DevOps

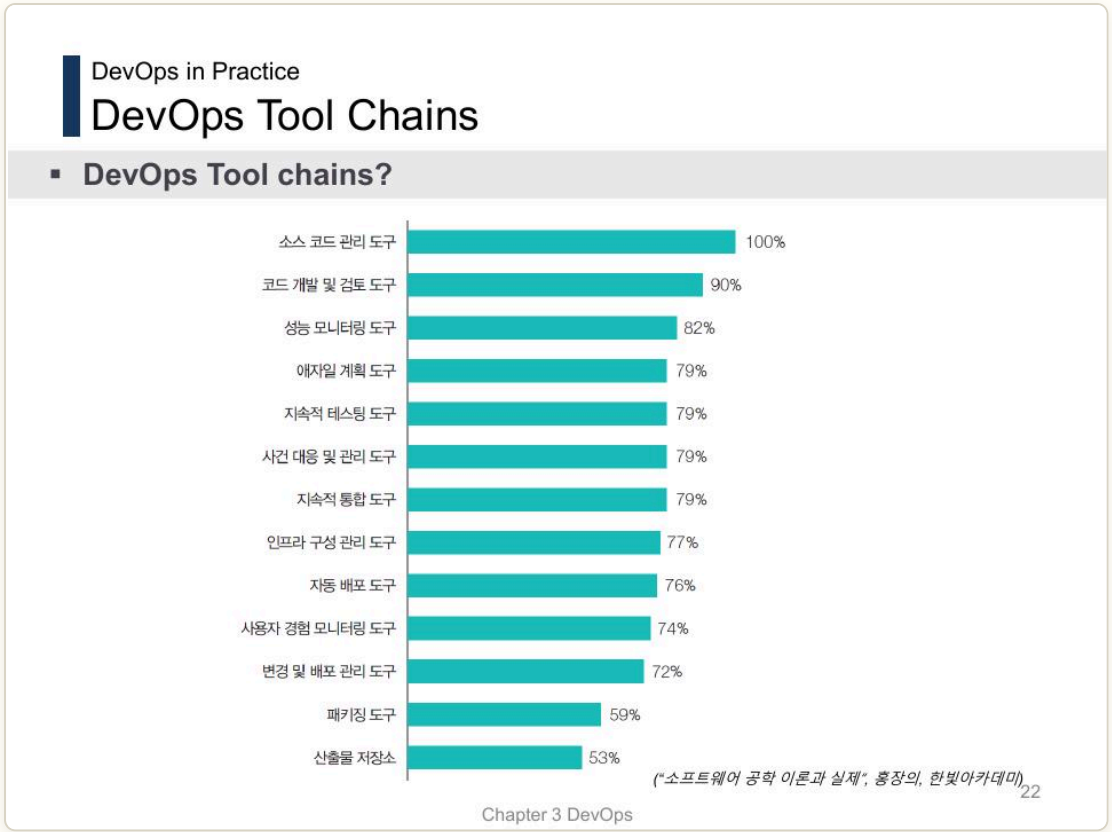
(https://www.padok.fr/en/blog/devops-process)

데브옵스 툴체인

- **DevOps toolchain** = 여러 공급사의 도구들이 **하나의 통합 단위**로 동작해 SW·시스템을 설계·빌드·테스트·관리·측정·운영
- 개발·운영팀이 제품 생명주기 전체에서 협업하게 하고
- 데브옵스 기본 — **지속적 통합, 지속적 전달, 자동화, 협업** — 을 다룬다

✍ 메모

★ 시험 툴체인 = 도구 모음이 하나로 연결된 것 (a collection of tools ... as an integrated unit)

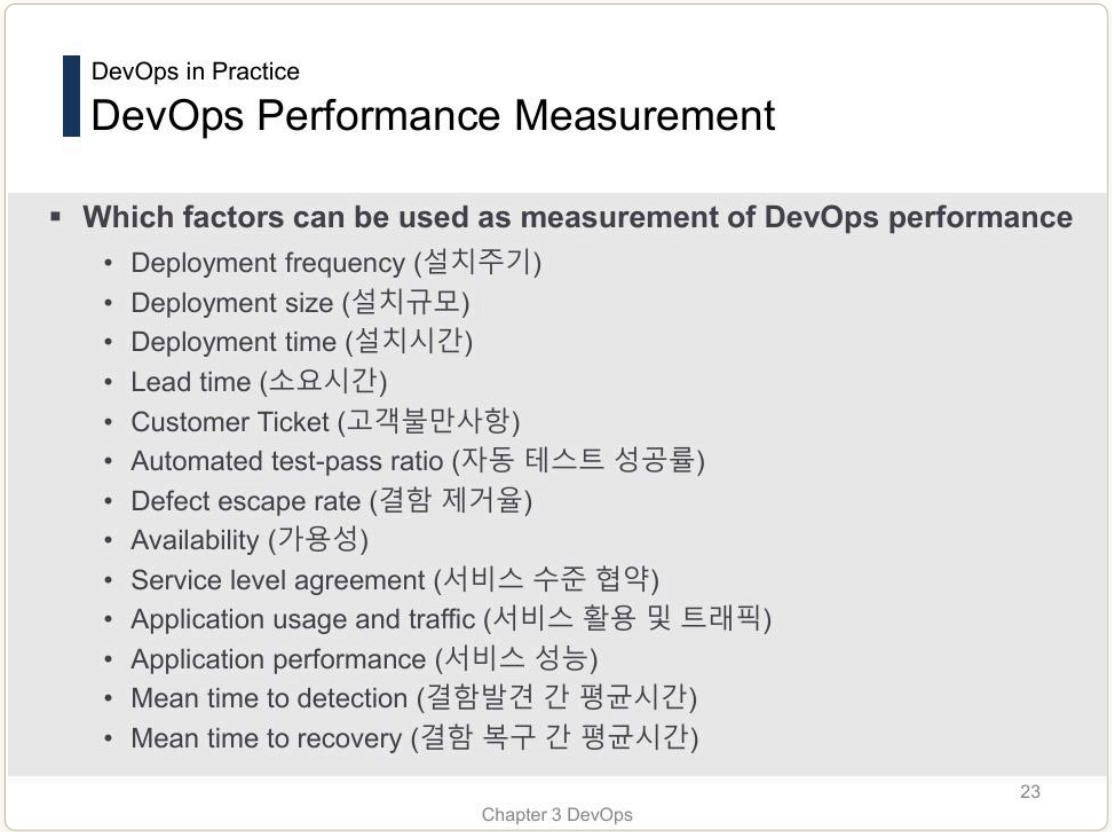


툴체인 도구 사용률 (그래프)

- 소스 코드 관리 100% · 코드 개발·검토 90% · 성능 모니터링 82%
- 애자일 계획 · 지속적 테스팅 · 사건 대응·관리 · 지속적 통합 각 79%
- 인프라 구성 관리 77% · 자동 배포 76% · 사용자 경험 모니터링 74% · 변경·배포 관리 72%
- 패키징 59% · 산출물 저장소 53%

✍️ 메모

💡 이해 1위 = 소스 코드 관리 도구(100%)



데브옵스 성과 측정 지표

- Deployment frequency (배포 주기)
- Deployment size (배포 규모)
- Deployment time (배포 시간)
- Lead time (소요 시간)
- Customer ticket (고객 불만 사항)
- Automated test-pass ratio (자동 테스트 성공률)
- Defect escape rate (결함 유출률)
- Availability (가용성)
- Service level agreement (서비스 수준 협약, SLA)
- Application usage and traffic (서비스 활용과 트래픽)
- Application performance (서비스 성능)
- **Mean time to detection (MTTD, 결함 발견까지 평균 시간)**
- **Mean time to recovery (MTTR, 결함 복구까지 평균 시간)**

✍️ 메모

⚠ 주의 슬라이드는 Defect escape rate를 '결함 제거율'로 적었지만, 뜻은 운영 환경으로 '빠져나간' 결함 비율(유출률)

💡 이해 슬라이드의 '설치주기·설치규모·설치시간' = 배포(deployment)



애자일과 데브옵스의 특징 (그림)

- **애자일**: 개인과 상호작용 / 동작하는 SW / 고객과의 협력 / 변화에 대응 (애자일 선언 4가지)
- **데브옵스**
  - **Systems thinking** — SW 애플리케이션이 복잡한 시스템임을 이해
  - **Amplifying feedback loops** — 팀원 사이 양방향 소통 개선
  - **Cultural change** — 지속적인 실험과 학습의 문화

✍️ 메모

★ 시험 데브옵스 3특징: 시스템 사고 · 피드백 루프 증폭 · 문화 변화

DevOps in Practice

DevOps vs. Agile

- **What is difference between DevOps and Agile?**

Agile	DevOps
Agile emphasizes collaboration between developers and product management	DevOps includes the operations team
Agile centers the flow of software from ideation to code completion	DevOps extends the focus to delivery and maintenance
Agile emphasizes iterative development and small batches	DevOps focuses more on test and delivery automation
Agile adds structure to planned work for developers	DevOps incorporates unplanned work common to operations teams

25

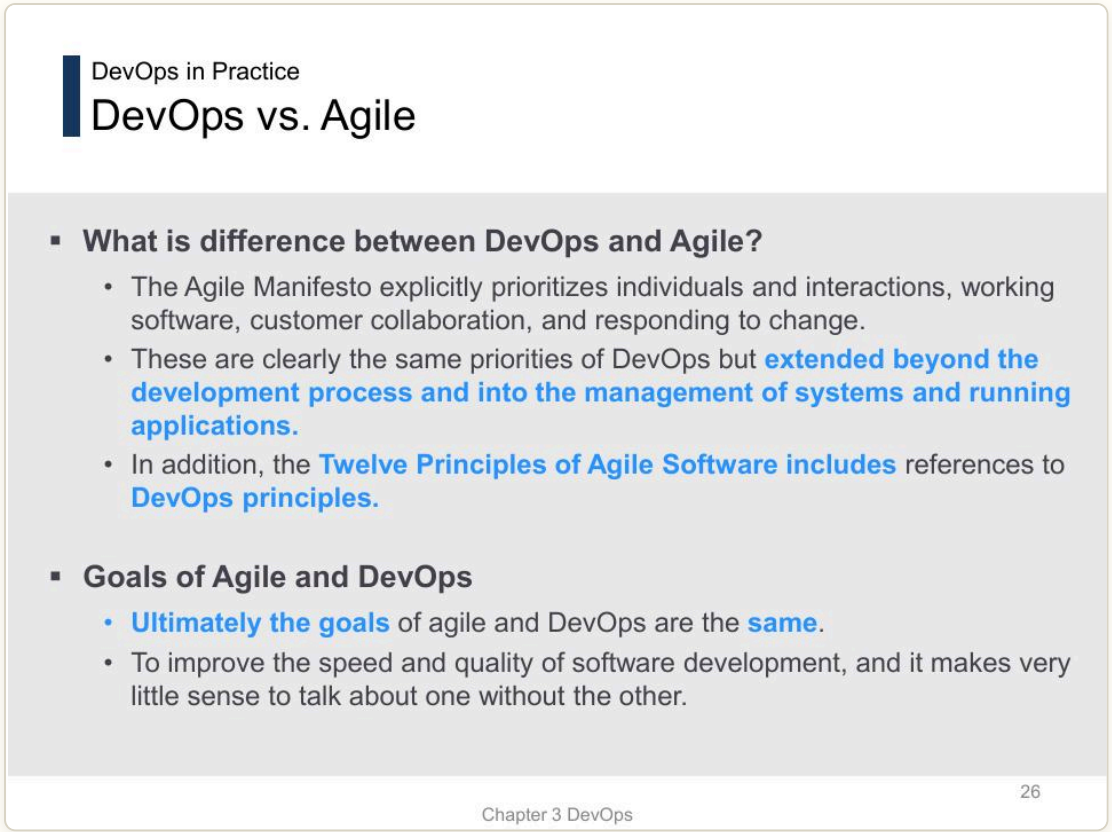
Chapter 3 DevOps

데브옵스 vs 애자일: 차이

- 애자일: 개발자와 **제품 관리** 사이 협업 강조 | 데브옵스: **운영팀**까지 포함
- 애자일: 아이디어에서 **코드 완성**까지의 흐름 | 데브옵스: **전달과 유지보수**까지 확장
- 애자일: **반복 개발과 작은 배치** 강조 | 데브옵스: **테스트·전달 자동화**에 더 집중
- 애자일: 개발자의 **계획된 일**에 구조를 더함 | 데브옵스: 운영팀에 흔한 **계획되지 않은 일**도 포함

✍ 메모

★ 시험 4쌍 비교표 — 서술형 1순위. 데브옵스 쪽 키워드: 운영팀 · 전달·유지보수 · 자동화 · 계획 외 업무

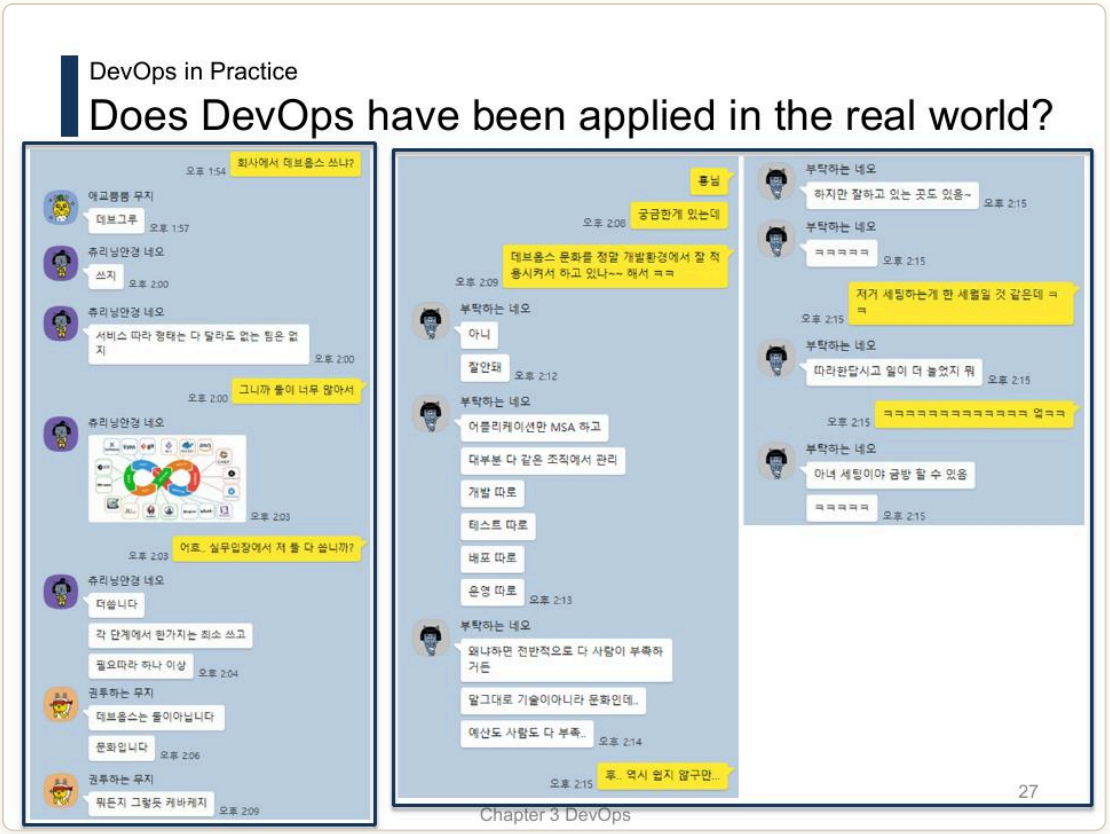


데브옵스 vs 애자일: 관계와 목표

- 애자일 선언은 개인과 상호작용, 동작하는 SW, 고객 협력, 변화 대응을 우선한다
  - 이는 데브옵스와 **같은 우선순위**지만, 데브옵스는 이를 개발 너머 **시스템 관리와 실행 중인 애플리케이션**까지 넓힌다
  - **애자일 12원칙**에도 데브옵스 원칙에 해당하는 내용이 있다
- **목표**: 결국 애자일과 데브옵스의 목표는 **같다**
  - SW 개발의 **속도와 품질 향상** — 하나를 빼고 다른 하나를 말하는 건 거의 의미가 없다

✍️ 메모

⚠ 주의 '데브옵스는 애자일을 대체한다/목표가 다르다' → X (목표 같음, 범위 확장)



현장에서 데브옵스는? (메신저 대화)

- 현업 개발자들에게 '회사에서 데브옵스 쓰나?' 물어본 대화 캡처
- 요지: 툴체인(CI/CD 등)은 많이 쓰지만, 진짜 어려운 건 도구보다 **문화**(개발·운영 협업)라는 이야기가 많다

✍️ 메모

💡 이해 p.15 '도입 장애 요인' 그래프와 같은 메시지 — 스킬·문화·사일로

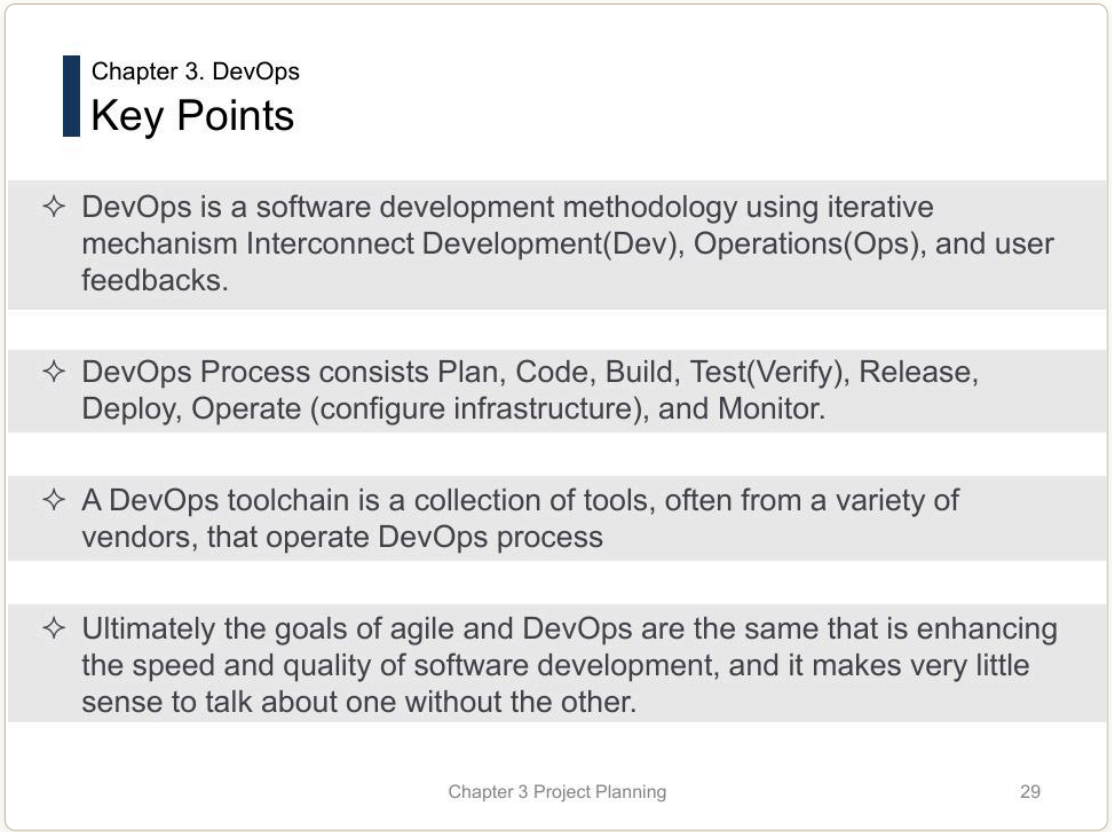


현장에서 데브옵스는? (메신저 대화)

- 현업 개발자들에게 '회사에서 데브옵스 쓰나?' 물어본 대화 캡처
- 요지: 툴체인(CI/CD 등)은 많이 쓰지만, 진짜 어려운 건 도구보다 **문화**(개발·운영 협업)라는 이야기가 많다

✍️ 메모

💡 이해 p.15 '도입 장애 요인' 그래프와 같은 메시지 — 스킬·문화·사일로



핵심 정리

- 데브옵스는 개발(Dev)·운영(Ops)·사용자 피드백을 서로 연결하는 반복적 SW 개발 방법론
- 데브옵스 프로세스: Plan, Code, Build, Test(Verify), Release, Deploy, Operate(인프라 구성), Monitor
- 데브옵스 툴체인 = 데브옵스 프로세스를 돌리는, 흔히 여러 공급사 도구의 모음
- 애자일과 데브옵스의 목표는 결국 같다 — SW 개발의 속도와 품질 향상, 하나 없이 다른 하나를 말하기 어렵다

✍️ 메모

— 넘어가도 되는 쪽